

Types of classes in Java

Page No. :-

Date :- / /

- Concrete class
- Abstract class
- Super class and sub class
- Object class
- Nested class
 - Inner class (Non static Nested class) ★
 - Anonymous Inner class
 - Member Inner class
 - Local Inner class
 - Static Nested class / static class
- Generic class ★
- POJO class
- Enum class
- Final class
- Singleton class
- Immutable class
- Wrapper class

Q what is concrete class?

- These are those class that we can create an instance using new keyword.
- All the methods in this class have implementation.
- It can also be your child class from interface or extend abstract class.
- A class access modifier can be "Public" or "package private" (no explicit modifier definition)


```

Ex. public class Person {
    int empId;
    Person(int empId) {
        this.empId = empId;
    }
    public int getId() {
        return empId;
    }
}

```

We can create of person class

```

Ex: public interface Shape {
    public void computeArea();
}

```

we cannot create an object of an interface.

```

public class Rectangle implements Shape {
    @Override
    public void computeArea() {
        sout("compute rec Area");
    }
}

```

always with help of interface
 with abstract class also
 but only when all methods are abstract. (0 to 100%)

2. Abstract class (0 to 100% Abstraction):

→ Show only important features to users and hide its internal implementation.

2 ways to achieve abstraction:

- class is declared as abstracted through keyword "abstract".
- It can have both abstract (method without body) and non abstract methods.
- We cannot create an instance of this class.
- We parent has some features which all child class have in common, then this can be used.

- Constructors can be created inside them. And with super keyword from child classes we can access them.

```

ex: public abstract class Car {
    int mileage;
    Car (int mileage) {
        this.mileage = mileage;
    }

```

```

    public abstract void pressBreak();
    public abstract void pressClutch();
    public int getNumberOfWheels() {
        return 4;
    }
}

```

Note: a child class who extend abstract class, has a responsibility to provide impl of abstract method otherwise declare himself as abstract class.

```

public abstract class LuxuryCar extends Car {
    LuxuryCar (int mileage) {
        super (mileage);
    }

```

add 1 more abstract method →

```

    public abstract void pressDualBreakSystem();
    @Override

```

provide defn of 1 parent abstract method but still 1 rem. →

```

    public void pressBreak() {
        //implem of it goes here
    }
}

```


concrete class → we can create obj of audi class

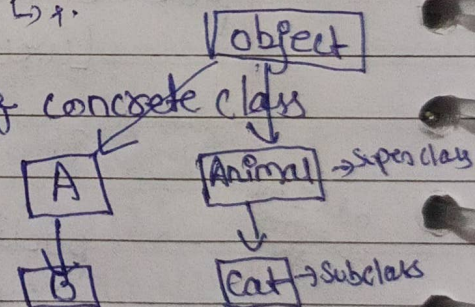
```
public class Audi extends LuxuryCar {
    Audi(int mileage) {
        super(mileage);
    }
    @Override
    public void pressClutch() {
        // its implemn goes here
    }
    @Override
    public void pressDualBreakSystem() {
        // its implemn goes here
    }
}
```

Note: We can use reference of abstract class to store child class obj. $\square = \text{new Car}()$ X Not allowed

Ex: $\text{Car audi} = \text{new Audi}(10);$

↑
abstract class

↑
store ref of concrete class



3. Super and Sub class:

- A class that is derived from another class is called Subclass.
- And from class through which subclass is derived it's called Superclass.
- In Java, in the absence of any other explicit Superclass, every class is implicitly a subclass of "Object class".
- Object is the topmost class in Java (Parent of all class). It has some common methods like clone(), wait(), toString(), equals(), notify(), notifyAll() etc.

Parent class ref = child class ref store in new child().
possible ✓

```
public class ObjectTest {
```

```
    psvm() {
```

```
        ObjectTest obj = new ObjectTest();
```

```
        Object obj1 = new Person(empID:1);
```

```
        Object obj2 = new Audi(mileage:10);
```

```
        sout(obj1.getClass()); // class Person
```

```
        sout(obj2.getClass()); // class Audi
```

4. Nested class :

→ class within another class is called Nested class.

- when to use?

→ If you know that, a class(A) will be used by only one another class(B), then instead of created new file (A.java) for it, we can create nested class inside B class itself.
And

Also help us to group logically related classes in one file.

Scope : Its Scope is same as of its Outer class.
Public/default (B) → [A] → has same scope as public/default.

→ It is of 2 types:

- Static Nested class

- Non static Nested class / Inner class

- Member inner class

- Local inner class

- Anonymous inner class

• Static Nested class:

- It do not have access to the non static instance variable and methods of outer class.
- Its object can be initiated without initiating the object of outer class.
- It can be private, public, protected or package-private (default, no explicit declⁿ)

Access modifier of outer class can either be public or default (private, protected & package-private are possible)

Ex: class OuterClass {

int instanceVar = 10;

static int classVar = 20;

Directly associated with class

static class NestedClass {

public void print() {

System.out.println(classVar + instanceVar);

Show error

Since it is instance (Not static)

but inner class can be

public, private, protected, default can be anything.

public class ObjectTest {

public static void main() {

OuterClass.NestedClass nestedObj =

new OuterClass.NestedClass();

nestedObj.print();

class with private access modifier:

- nested class obj can be created within the same class itself.

ex:

```
class OuterClass {
```

```
    int instanceVar = 10;
```

```
    static int classVar = 20;
```

```
        private static class NestedClass {
```

```
            public void print() {
```

```
                sout(classVar);
```

```
            }
```

```
        public void display() {
```

```
            NestedClass nestedObj = new NestedClass();
```

```
            nestedObj.print();
```

```
        }
```

```
    }
```

```
public class ObjectTest {
```

```
    psum() {
```

```
        OuterClass outerObj = new OuterClass();
```

```
        outerObj.display();
```

```
    }
```

```
}
```


Inner class or Non Static Nested class:

- It have access to all the instance variable and method of outer class.
- Its object can be initiated on after initiating the object of outer class.

1. Member Inner class:

- Its can be private, public, protected, default.

```
class OuterClass {
```

```
    int instanceVar = 10;
```

```
    static int classVar = 20;
```

// Since class is not static so it invoke only with obj of OuterClass obj

```
    class InnerClass {
```

```
        public void print() {
```

```
            sout(classVar + instanceVar);
```

```
        }
```

```
    }
```

```
}
```

```
public class ObjectTest {
```

```
    psvm() {
```

```
        OuterClass outerClassObj = new OuterClass();
```

```
        OuterClass.InnerClass innerClassObj =
```

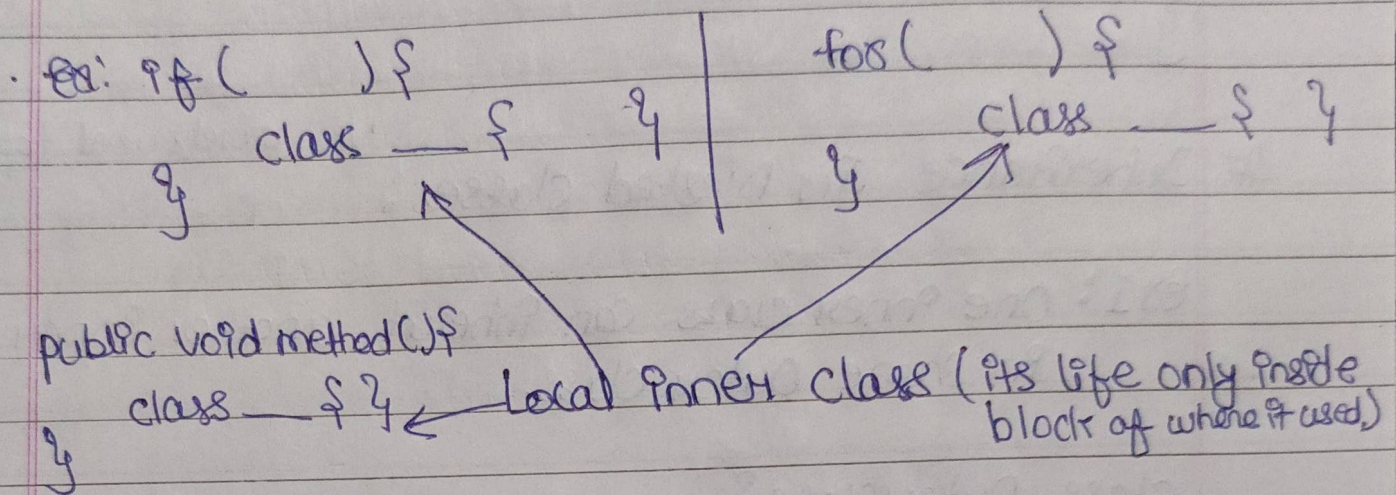
```
            outerClassObj.new InnerClass();
```

```
        innerClassObj.print();
```

2. Local Inner class:

- These are those classes which are defined in any block like for loop, while loop block, if condition block, method etc.

- It can not be declared as private, protected, public only default (not defined explicit) access modifier is used.
- It can not be initiated outside of this block.



Ex: `class OuterClass {`

`int instanceVar = 1;`

`static int classVar = 2;`

`public void display() {`

`int methodLocalVar = 3;`

`class LocalInnerClass {`

`int localInnerVar = 4;`

`public void print() {`

`System.out.println(instanceVar + classVar + methodLocalVar + localInnerVar);`

`LocalInnerClass localObj = new LocalInnerClass();`

`localObj.print();`

Page No. :
 Date : / /

```

public class ObjectTest {
    psum() {
        OuterClass outerClassObj = new OuterClass();
        outerClassObj.display();
    }
}
  
```

Inheritance in Nested classes: very rarely used topic

Ex 1: One inner class can inherit another inner class in same outer class.

```

class OuterClass {
    int instanceVar = 1;
    static int classVar = 2;
    class InnerClass {
        int innerClass1 = 3;
    }
    class InnerClass2 extends InnerClass {
        int innerClass2 = 4;
        void display() {
            sout(innerClass1 + innerClass2 +
                instanceVar + classVar);
        }
    }
}
  
```

```

public class ObjectTest {
    psum() {
        OuterClass outerClassObj = new OuterClass();
        outerClassObj.InnerClass2 pObj2 = new InnerClass2();
        pObj2.display();
    }
}
  
```